

聚合-组合

继承不是类代码复用的唯一方式

类之间除了继承关系外，还存在一种整体与部分的关系。

1. 聚合

- 在聚合关系中，被包含的对象与包含它的对象独立创建和消亡，被包含的对象可以脱离包含它的对象独立存在。
 - 例如，一个公司与它的员工之间是聚合关系。
- 实现上，聚合类的成员对象一般是采用**对象指针**表示，用于指向被包含的成员对象，而被包含的成员对象是在**外部**创建，然后加入到聚合类对象中来的。

```
1  class A { ..... };
2  class B //B与A是聚合关系
3  { A *pm; //指向成员对象
4  public:
5  B(A *p) { pm = p; } //成员对象在聚合类对象外部创建，然后传入
6  ~B() { pm = NULL; } //传进来的成员对象不再是聚合类对象的成员
7  .....
8  };
9  .....
10 A *pa=new A; //创建一个A类对象
11 B *pb=new B(pa); //创建一个聚合类对象，其成员对象是pa指向的对象
12 .....
13 delete pb; //聚合类对象消亡了，其成员对象并没有消亡
14 ..... // pa指向的对象还可以用在其它地方
15 delete pa; //聚合类对象原来的成员对象消亡
16
```

例子：公司和职员

2. 组合

- 在组合关系中，被包含的对象随包含它的对象创建和消亡，被包含的对象不能脱离包含它的对象独立存在。
 - 例如，一个人与他的头、手和脚之间则是组合关系。
- 实现上，组合类的成员对象一般直接是对象，有时也可以采用对象指针表示，但不管是什么表示形式，成员对象一定是在组合类对象内部创建并随着组合类对象消亡。

```
1  class A
2  { .....
3  };
4  class C //C与A是组合关系
5  { A a;
6  public:
7  .....
8  };
9  .....
10 C *pc=new C; //创建一个组合类对象，其成员对象在组
```

```
11 合类对象内部创建
12 .....
13 delete pc; //组合类对象与其成员对象都消亡了
14
```

3.继承与聚合/组合的比较

- 继承与封装存在矛盾，聚合/组合与封装则不存在这个矛盾。
- 在基于继承的代码复用中，一个类向外界提供两种接口：
 - public：对象（实例）用户
 - public+protected：派生类用户
- 在基于聚合/组合的代码复用中，一个类对外只需一个接口：public。

继承的代码复用常常可以用组合来实现。

- 继承更容易实现子类型：
 - 在C++中，public继承的派生类往往可以看成是基类的子类型。
 - 在需要基类对象的地方可以用派生类对象去替代。
 - 发给基类对象的消息也能发给派生类对象。
- 具有聚合/组合关系的两个类不具有子类型关系！